

In one line: Everything you ship — JS bundle, Hermes bytecode, Info.plist, BuildConfig — is **readable by anyone who downloads the app**, and on a rooted/jailbroken phone the attacker also **controls your process at runtime**. So: no secrets in the client, tokens only in **Keychain/Keystore**, raise the cost (pinning, R8, RASP), and let the **server** decide.

Where can a value live — and who can read it?	store download: unzip IPA / APK	device in hand / backup / adb	root / jailbreak + Frida at runtime	so use it for
JS bundle / .hbc: literals, EXPO_PUBLIC_*, RN-config	reads	reads	reads	public config: API URL, publishable keys (app-restricted)
native: Info.plist, BuildConfig, strings.xml	reads	reads	reads	identifiers only — “hidden in native” is still in the package
AsyncStorage, plain MMKV, files	—	reads (backup)	reads	non-secret prefs, caches, feature flags
MMKV encrypted, key held in the Keychain	—	ciphertext only	hooks the key	larger sensitive data (cached profile, drafts)
Keychain / Keystore, _ThisDeviceOnly	—	—	uses; can't export	refresh token, the MMKV key, private keys
JS memory (a variable)	—	—	reads	short-lived access token
your server / KMS	—	—	—	client secrets, 3rd-party API keys, signing keys, the rules

EAS “secret” / CI secret = readable only by the build job — expose it to JS (EXPO_PUBLIC_ prefix, extra in app.config) and it is in the bundle: row 1. Each column can also do everything to its left.

How the bundle leaks

- Release iOS embeds main.jsbundle in the .app; Android ships assets/index.android.bundle. With Hermes it is **bytecode** — compiled for start-up speed, not secrecy: strings lists every literal, and decompilers (hermes-dec, hbctool) rebuild readable functions.
- Minification renames *locals*; string literals, object keys, URLs and GraphQL documents survive. JS obfuscators only raise the cost.
- R8** (def enableProguardInReleaseBuilds = false in the template — flip it) shrinks + obfuscates Java/Kotlin only, never JS; reflection-using libs need keep rules; upload mapping.txt.
- Template manifest: android:allowBackup="false"; cleartext only via a build-time usesCleartextTraffic placeholder.

Storage — pick by what it protects

- react-native-keychain:** set/get/resetGenericPassword; ACCESS_CONTROL.BIOMETRY_CURRENT_SET; Android STORAGE_TYPE AES_GCM_NO_AUTH · AES_GCM (biometric) · AES_CBC deprecated.
- expo-secure-store:** iOS Keychain kSecClassGenericPassword (default WHEN_UNLOCKED); Android SharedPreferences encrypted by a Keystore key; requireAuthentication; small values (iOS historically ~2KB).
- react-native-mmkv** v4: createMMKV({ id, encryptionKey }); AES-128 (AES-256 opt.), sync JSI, RN ≥ 0.76. A key hard-coded in JS = no encryption.
- AsyncStorage:** plain text — per the RN docs, never tokens or secrets. Jetpack EncryptedSharedPreferences is **deprecated** (security-crypto 1.1.0).

Example — encrypted MMKV, key in the Keychain

```
import * as Keychain from 'react-native-keychain';
import { createMMKV } from 'react-native-mmkv';
export async function openVault() {
  const svc = { service: 'vault-key' };
  const hit = await Keychain.getGenericPassword(svc); // false if none
  const k = hit ? hit.password : randomKey(); // CSPRNG (expo-crypto)
  if (!hit) await Keychain.setGenericPassword('vault', k, { ...svc,
    accessible: Keychain.ACCESSIBLE.AFTER_FIRST_UNLOCK_THIS_DEVICE_ONLY });
  return createMMKV({ id: 'vault', encryptionKey: k }); // key never in JS source
}
```

Root / jailbreak detection — cost, not a wall

jail-monkey (isJailBroken()), expo-device isRootedExperimentalAsync(), **freeRASP** (Talsec; root, hooks, tampering, untrusted installs). All are heuristics (e.g. su paths) that hiding and hooking tools (Magisk + Shamiko, Frida) defeat, and the result is a **JS boolean** — one hook flips it. Use them to *signal* risk; the server decides with **App Attest / Play Integrity** verdicts.

Pinning — three ways in RN

- react-native-ssl-public-key-pinning: TrustKit (iOS) + OkHttp CertificatePinner (Android); covers fetch/XHR, so axios too; initializeSslPinning({ 'api.example.com': { publicKeyHashes: [...] } }) — base64 SHA-256 of the SPKI, ≥ **2 pins** (TrustKit enforces it). Pins set from JS ⇒ an OTA update can rotate them.
- Android only, no code: network_security_config.xml <pin-set expiration="yyyy-MM-dd"> + backup <pin digest="SHA-256">; after expiration pinning switches off (fail-open by design).
- Android ≥ 7 (targetSdk 24+) ignores **user-installed CAs** — that is why Charles/Proxyman needs <debug-overrides>; don't ship src="user".

OTA update integrity — expo-updates code signing

HTTPS alone trusts the CDN/host that serves the JS. npx expo-updates codesigning:generate; updates.codesigningCertificate + codeSigningMetadata { keyid, alg: "rsa-v1_5-sha256" }; eas update --private-key-path. The client checks the signature against the **embedded cert** and **rejects** anything else (EAS Production/Enterprise plans). New cert = store build; the private key stays off the repo.

Traps

- ! “Hermes compiles it, so the key is safe” — strings app.hbc | grep sk_.
- ! “It’s an EAS secret” — secret from people, not from the bundle it’s inlined in.
- ! Token persisted by redux-persist to AsyncStorage, or sent in a Sentry breadcrumb.
- ! Root check or pinning as the only control: the attacker owns the process — authZ, rate limits, prices, entitlements live on the server.
- ! iOS Keychain survives uninstall: a reinstall inherits the old refresh token.

Remember

The binary is public · the device may be hostile · the server decides.